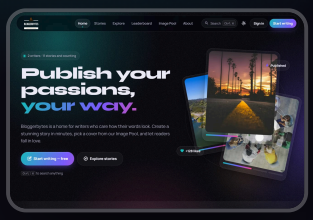
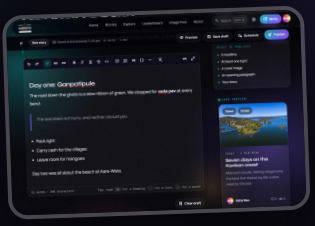
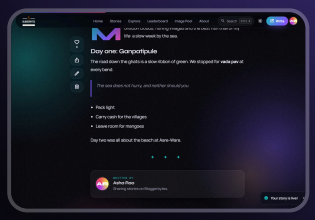
PROJECT REPORT · VERSION 2.0 · 2026

Bloggerbytes

A full-stack blogging platform with a rich-text studio, drafts and scheduled publishing, on the web and on Android and iOS.



AUTHOR

Manjunatha M S

ORIGINAL PROJECT

Mini project, B.E. Computer Science & Engineering, 2018–19

STACK

Angular 22 · Node.js / Express 5 · MongoDB · Flutter 3.47

REPOSITORY

github.com/mrmanjunathams/Web_Tech

Abstract

Writing and reading blogs is one of the simplest ways to share knowledge. Bloggerbytes is a place where anyone can sign up, write a story with a cover picture from a curated Image Pool, and publish it for readers who can explore, search, like and share it.

Bloggerbytes began in 2018–19 as a seventh-semester mini project: static HTML pages styled with Materialize CSS, an AngularJS login form and an Express server writing to a hosted MongoDB. This report describes version 2.0, a complete rebuild of the same idea as a modern, secure and fully tested product that runs in the browser and as a native mobile app.

The rebuilt system keeps every page of the original — Home, Blog, Categories, Rewards, Image Pool, About Us, My Account and Write Blog — and adds what a writer expects today: a rich-text editor with headings, lists, quotes, code and links; drafts that save themselves; scheduled publishing; editing of published stories; display names; likes, views and a leaderboard; instant search; and light and dark themes. A Flutter application brings the same features to Android and iOS using the same API.

Security was a major goal. The original source code contained the database and e-mail passwords; version 2.0 keeps secrets out of the code, hashes passwords with scrypt, uses signed HTTP-only session cookies with rate-limited sign-in, and sanitizes every story body on the server against a strict allowlist.

Quality is enforced automatically: the API, the web app and the mobile app each have a test suite with 100% code coverage, plus a contract test that runs the mobile app's API layer against the real server. GitHub Actions run all of this on every push and build the deployable API package, the web app, an Android APK and a web build of the mobile app.

AUTOMATED TESTS

204

48 API · 81 web · 66 mobile · 9 live-API contract checks

CODE COVERAGE

100%

Lines in all three parts; statements, branches and functions too for API and web

PLATFORMS

3

Web (any browser), Android and iOS, one API

Table of Contents

Abstract 2

List of Figures 4

1 Introduction 5

1.1 Background · 1.2 Objectives · 1.3 Technologies used

2 Literature Survey 7

3 Problem Definition 8

4 System Requirements 9

5 System Design 10

5.1 Architecture · 5.2 Data model · 5.3 API · 5.4 Story life cycle · 5.5 Rich text · 5.6 Security

6 Implementation 14

6.1 Modules · 6.2 Writing studio · 6.3 Mobile app · 6.4 Code highlights

7 Testing and Quality Assurance 17

8 Snapshots 19

9 Conclusion and Future Enhancements 29

Bibliography 30

List of Figures

Figure	Title
5.1	System architecture of Bloggerbytes 2.0
5.2	Navigation flow between the pages
5.3	Life cycle of a story: draft, scheduled, published
5.4	Rich-text pipeline from editor to reader
7.1	Continuous integration on GitHub Actions
8.1 – 8.14	Web application: home, stories, explore, sign-up, studio, preview, scheduling, reader, dashboard, leaderboard, Image Pool, About, phone layout
8.15 – 8.20	Mobile application: home, explore, studio, reader, dashboard, leaderboard

CHAPTER 1

Introduction

1.1 Background

A blog is a regularly updated collection of articles written by one person or a community. Blogging platforms make publishing easy: a writer focuses on words and pictures, and the platform handles layout, storage and discovery. Bloggerbytes is such a platform, built around a simple idea from the original project: *every story deserves a beautiful cover*, so writers pick a photograph from a curated Image Pool instead of uploading their own.

The first version (2018–19) delivered the pages and the basic flow — sign up, log in, write a two-paragraph blog and see it listed by category. It was a learning project: credentials were written into the source, blogs were plain text, there was no way to edit or delete a post, and nothing was tested. Version 2.0 rebuilds the same product with current tools and engineering practice.

1.2 Objectives

- Keep every page and the visual identity of the original site, re-implemented with a modern design system (glass surfaces, gradient accents, light and dark themes).
- Give writers a real editor: formatted text, headings, lists, quotes, code and links — with drafts that save themselves, scheduled publishing and the ability to edit or unpublish.
- Show people by their name, not their login: a display name chosen at sign-up and editable later.
- Reach readers on phones with a native mobile app that shares the same API.
- Make the system secure by design and keep it working through automated tests and continuous integration.

1.3 Technologies used

Layer	Technology	Role in Bloggerbytes
Web front end	Angular 22, TypeScript 6	Standalone components, signals and zoneless change detection; lazy-loaded pages; hash routing so page URLs never clash with API paths.
Styling	CSS custom properties, Bootstrap 5.3 grid	A single design system ("Aurora Editorial") with light/dark tokens, re-creating the original Materialize look.
Rich-text editor (web)	TipTap 3 (ProseMirror)	WYSIWYG editing with a toolbar, keyboard shortcuts and Markdown-style typing shortcuts.
API server	Node.js 22/24, Express 5	JSON REST API, cookie sessions, rate limiting, serving the built web app.

Database	MongoDB, Mongoose 9	Users, blogs (with likes, views, status and schedule) and newsletter subscribers; sessions stored with connect-mongo.
Sanitizer	sanitize-html	Allowlist-based cleaning of every story body before it is stored.
Mobile app	Flutter 3.47, Dart 3.13	Android, iOS and web from one code base; go_router navigation; flutter_quill editor; HTML rendering of stories.
Testing	node:test + supertest, Vitest + jsdom, flutter_test	Unit, component, widget and contract tests with coverage gates.
Automation	GitHub Actions	Test, check coverage and build on every push.

Angular is the successor of AngularJS used in the original project. It organises the interface into components and services; version 22 adds *signals*, a reactive primitive that updates exactly the parts of the page that depend on changed data. **Express** remains the server framework, now in version 5 with built-in handling of asynchronous errors. **MongoDB** stores documents as JSON-like records, which suits blogs whose shape evolves — for example, version 2.0 added a status and a schedule to blogs without breaking stories written in 2018. **Flutter** is Google's toolkit for building natively compiled apps for mobile, web and desktop from a single code base in the Dart language.

CHAPTER 2

Literature Survey

The original report surveyed general web-development texts. Version 2.0 builds on those foundations and on the official documentation and security guidance of the tools it uses.

[1] Randy Connolly, Ricardo Hoar — "Fundamentals of Web Development"

Covers client- and server-side web development: HTML, CSS, JavaScript, HTTP and server frameworks. Its model of a web application — a browser client talking to a server that owns the data — is still the backbone of Bloggerbytes; version 2.0 adds a second client (the mobile app) on the same API.

[2] OWASP Top 10 and the OWASP Cheat Sheet Series

The Open Worldwide Application Security Project lists the most critical web risks. Three shaped the design directly: *injection* (cross-site scripting through user-written HTML, prevented by server-side allowlist sanitization), *broken authentication* (prevented with salted scrypt hashes, session regeneration on sign-in and rate limiting), and *security misconfiguration / sensitive data exposure* (secrets moved from source code into environment variables).

[3] Angular documentation (angular.dev)

Guides on standalone components, signals, zoneless change detection and the router informed the front-end architecture — notably that a routed component is reused when only the query string changes, which the studio handles explicitly.

[4] ProseMirror and TipTap documentation

ProseMirror models a document as a schema-checked tree rather than raw HTML, which makes a rich-text editor predictable. TipTap packages it with ready-made extensions for headings, lists, links and character counting.

[5] Flutter documentation (docs.flutter.dev) and Material Design 3

Describe widget composition, navigation, testing with `flutter_test`, and the Material 3 component set used across the mobile app.

[6] Express, Mongoose and MongoDB manuals

Reference for routing, middleware ordering, schema validation, aggregation pipelines (used for statistics and the leaderboard) and query filters such as the "live story" filter that hides drafts and not-yet-due scheduled stories.

[7] W3Schools and MDN Web Docs

As in the original project, these remained handy references for HTML, CSS and browser APIs such as the Web Share API, the Clipboard API and `prefers-color-scheme`.

CHAPTER 3

Problem Definition

The goal is a blog reading and writing platform where a writer can sign up, compose a well-formatted story with a cover from the Image Pool, keep it as a draft or publish it now or later, and where readers can browse by topic, search, read, like and share — on the web and on phones. The project also contains a rewards section where the most-loved writers are recognised.

Limitations of the original version

Problem in 2018–19	How version 2.0 solves it
Database and e-mail passwords written in the source code	Secrets live only in `backend/.env` (ignored by git); production refuses to start without a session secret.
Blogs were two plain paragraphs	Rich-text editor (web and mobile) producing sanitized HTML; old plain-text stories are converted to paragraphs automatically.
No drafts, no editing, no deleting	Server-side drafts with autosave, scheduled publishing, edit/update, move back to drafts and delete.
People shown by their login name	A display name chosen at sign-up, shown everywhere and editable from the dashboard.
Rewards were a static page	A live leaderboard ranked by likes, posts and views.
Web only, desktop-first	Responsive web app plus a Flutter app for Android and iOS.
No tests or automation	Three test suites at 100% coverage, a live contract test and CI on every push.

CHAPTER 4

System Requirements

4.1 Hardware requirements

Component	Development machine	Server (production)	Client
Processor	Any 64-bit dual-core, 2 GHz+	1 vCPU	Any modern phone or computer
Memory	8 GB RAM (16 GB with Android emulator)	512 MB – 1 GB	—
Storage	10 GB free (SDKs and dependencies)	1 GB + database	~60 MB for the Android app

4.2 Software requirements

Software	Version / notes
Operating system	Windows 10/11, macOS or Linux (development); Linux (server)
Runtime	Node.js 22.22+ or 24 LTS
Database	MongoDB 7+ (local, Atlas, or the bundled in-memory server for development)
Web client	Any current browser (Chrome, Edge, Firefox, Safari)
Mobile	Flutter 3.47 SDK; Android 7+ / iOS 13+ devices; Android SDK 36 and JDK 17 to build
Editor	Visual Studio Code (any editor works)
Version control and CI	Git and GitHub Actions

CHAPTER 5

System Design

5.1 Architecture

Bloggerbytes follows a client–server architecture. A single Express API owns all data and rules; the Angular web app and the Flutter mobile app are two clients of it. In production the API also serves the built web app, so the browser talks to a single origin and the session cookie just works.

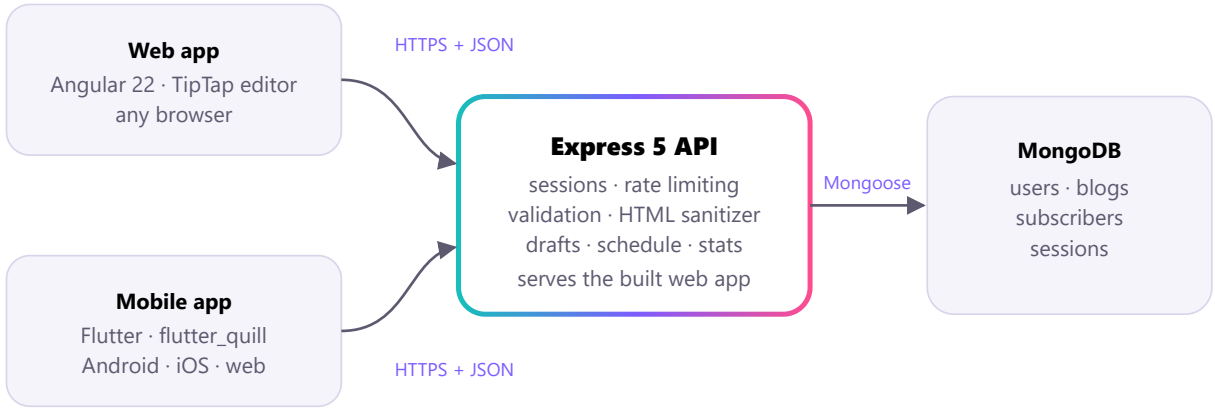


Figure 5.1 — System architecture of Bloggerbytes 2.0

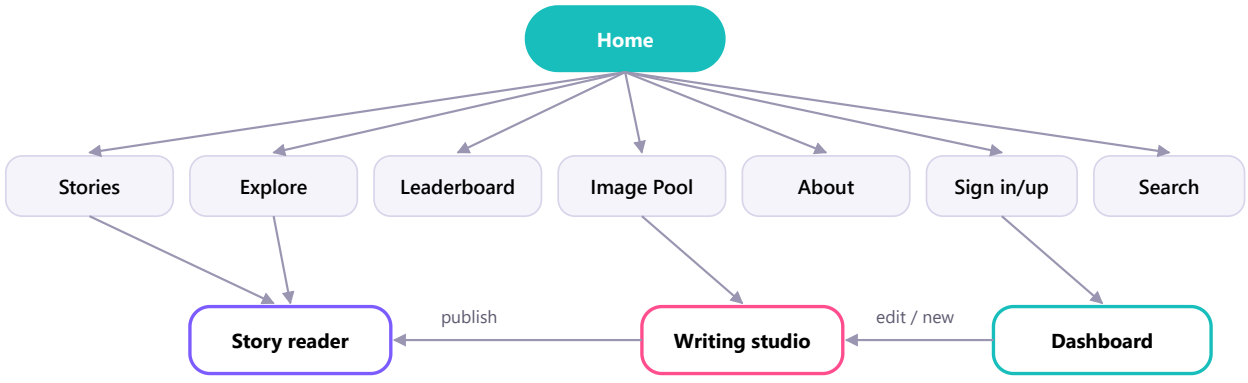


Figure 5.2 — Navigation flow between the pages (the same routes exist in the web and mobile apps)

5.2 Data model

Collection	Field	Purpose
users	`username` (unique)	Login identity, 3–30 letters, digits, `. _ -`
	`name`	Display name shown on stories (1–60 characters)
	`email` (unique)	Contact and welcome e-mail
	`passwordHash`	scrypt hash with a random salt; never returned by the API
blogs	`auth`	Author's username (taken from the session, never from the request)
	`title` , `cate` , `img`	Headline, topics (`"Travel | Ocean"`), Image Pool picture name
	`blogp1`	Opening paragraph (plain text, also the card excerpt)
	`blogp2`	Story body as sanitized HTML
	`status` , `publishAt`	Draft or published; time the story goes live (future = scheduled)
	`date`	Display date such as `01-NOV-2026`
	`likedBy` , `views`	Who liked it (never exposed, only the count) and view counter
subscribers	`email` (unique)	Newsletter sign-ups

5.3 API

Method	Path	Auth	Description
GET	`/BAdd`	—	Live stories, newest first (drafts and future schedules hidden)
GET	`/blogs`	—	Featured editor's picks
GET	`/posts/:id`	—	One story; counts a view; drafts only for their author
POST	`/addblog`	signed in	Publish now, schedule (`publishAt`) or save a draft (`status: "draft"`)
PATCH	`/posts/:id`	author	Edit content, publish, unpublish or reschedule
DELETE	`/posts/:id`	author	Delete a story
POST	`/posts/:id/like`	signed in	Toggle a like

POST	`/adduser`, `/loginuser`, `/logout`	—	Sign up (with name), sign in, sign out (rate limited)
GET / PATCH	`/myacc`	signed in	Profile with all own stories / change display name
GET	`/stats`, `/leaderboard`	—	Site totals and the top ten writers
POST	`/subscribe`	—	Newsletter sign-up

5.4 Story life cycle

A story is visible to the public when its status is *published* and its publish time has passed. Scheduling therefore needs no background job: a scheduled story simply becomes "live" when the clock reaches its `publishAt`, because every public query applies the same live filter.

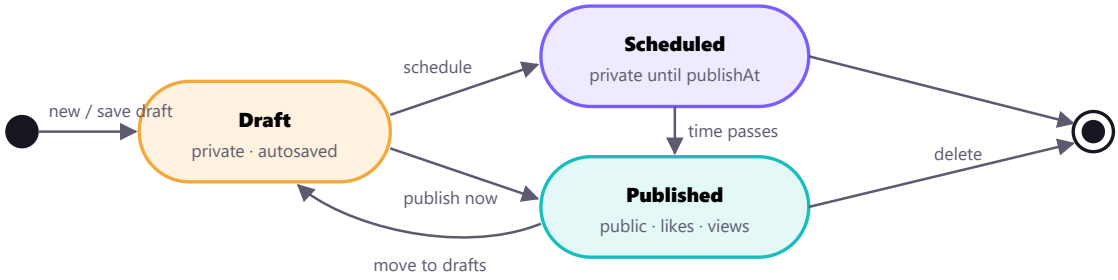


Figure 5.3 — Life cycle of a story: draft, scheduled, published

5.5 Rich text

Both editors produce HTML. The web uses TipTap (ProseMirror); the mobile app uses flutter_quill and converts its document to and from HTML. The server is the single gatekeeper: it sanitizes every body against an allowlist of tags (`p, h2, h3, strong, em, u, s, code, pre, blockquote, ul, ol, li, a, hr, br`), keeps only `http`, `https` and `mailto` links, and forces `rel="noopener noreferrer nofollow"`. Stories written in 2018 as plain text are turned into paragraphs on the way out, so every client can always render HTML.

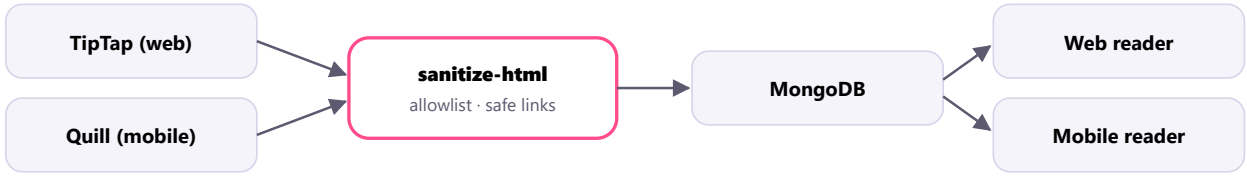


Figure 5.4 — Rich-text pipeline from editor to reader

5.6 Security design

- **Passwords** are hashed with scrypt and a 16-byte random salt and compared in constant time.

- **Sessions** use a signed, HTTP-only `bb.sid` cookie (secure and `SameSite` in production), regenerated at sign-in to prevent session fixation; sign-in and sign-up are rate limited.
- **Authorisation**: the author of a story always comes from the session; only the author can edit, delete or see a draft.
- **Input**: every field is validated and length-limited; operator objects such as `{"$gt": ""}` are rejected; request bodies are capped at 512 KB.
- **Output**: password hashes and the list of likers are never sent; story HTML is sanitized on the server and again by Angular when displayed.
- **Configuration**: secrets live only in environment variables; Helmet sets secure HTTP headers; CORS is off unless specific origins are configured.

CHAPTER 6

Implementation

The main modules of the original project are all present and have grown into the following.

6.1 Modules

Module	What it does
Home	Index page with live statistics, a rotating headline, trending stories (most liked, then most viewed), popular topics, feature highlights and a call to action. The navigation bar gives access to every page, search and the theme switch; the footer has the newsletter and social links.
Stories (Blog)	Editor's picks in a magazine layout plus the latest community posts.
Explore (Category)	All live stories with instant search across title, author name, topics, date and story text; topic chips with counts; sorting by newest, most loved, most viewed or A–Z.
Story reader	Cover, author, reading time, word count and views; rich-text body; reading progress bar; like with an animated heart; share (Web Share or copy link); edit and delete for the author; related stories.
Leaderboard (Rewards)	Podium of the top three writers and a ranking list, ranked by likes received, then posts and views; badges to aim for.
Image Pool	Gallery of nineteen photographs with a full-screen viewer (keyboard and swipe) and "Write with this" to start a story with a chosen cover.
About Us	The mission, the project's history as a timeline and contact details.
Sign up / Sign in	Name, username, e-mail and password with live validation and a password-strength meter.
Dashboard (My Account)	Profile with an editable display name; totals for stories, likes, views and words; stories organised into Published, Scheduled and Drafts tabs with edit and delete.
Writing studio (Write Blog)	Described in section 6.2.

6.2 Writing studio

The studio was redesigned around the writing itself. A sticky bar shows the story's state (**New story** **Draft** **Scheduled** **Published**), whether changes are saved, the word count and reading time, and the actions that make sense for that state. The canvas holds the cover, the headline, the opening paragraph and the rich-text editor; a side panel holds topics with suggestions, a publishing checklist and a live preview of the story card.

- **Editor toolbar**: undo/redo; paragraph, heading and subheading; bold, italic, underline, strikethrough, inline code; bulleted and numbered lists, quote, code block, divider; links; clear formatting; a distraction-free focus mode.
- **Typing shortcuts**: `##` heading, `-` list, `1.` numbered list, `>` quote, ``` ``` code block, `**bold**`; `Ctrl+K` link, `Ctrl+S` save.
- **Drafts**: a new story is kept in the browser (or on the phone) until its first save; after that it autosaves to the server two seconds after the last change. Leaving the page saves pending changes first.
- **Publishing**: publish now, or schedule a date and time up to a year ahead. Published and scheduled stories change only when you press *Update*, and leaving with unsaved changes asks for confirmation. Any story can be moved back to drafts or deleted.

6.3 Mobile application

The Flutter app covers every page of the web app with native navigation: a bottom bar for the main sections, a drawer for the full menu, an account menu and a floating *Write* button. It uses the same API and the same session cookie, which it stores securely on the device so the writer stays signed in. Pictures from the Image Pool ship inside the app for instant loading, and the build checks that they match the web copy.

The mobile studio offers the same features as the web: a Quill-based rich-text editor with a formatting toolbar, cover picker, topics, checklist and card preview, preview mode, device-side and server-side drafts with autosave, publish now, schedule with native date and time pickers, update, move to drafts and delete.

6.4 Code highlights

Which stories the public can see

```
/** Published, and live (no schedule, or the schedule has passed). */
const liveFilter = () => ({
  status: { $ne: 'draft' },
  $or: [{ publishAt: null }, { publishAt: { $lte: new Date() } }],
});
```

Sanitizing a story body

```
const SANITIZE = {
  allowedTags: ['p', 'br', 'h2', 'h3', 'strong', 'em', 'u', 's', 'code', 'pre',
                'blockquote', 'ul', 'ol', 'li', 'a', 'hr'],
  allowedAttributes: { a: ['href', 'target', 'rel'] },
  allowedSchemes: ['http', 'https', 'mailto'],
  transformTags: { h1: 'h2', b: 'strong', i: 'em',
    a: (tag, a) => ({ tagName: tag, attribs: { href: a.href || '', target: '_blank',
                                               rel: 'noopener noreferrer nofollow' } }) },
};
```

Saving drafts automatically (web studio)

```
effect(() => {
  const fields = this.fields();
  if (!this.dirty() || !untracked(this.autosaves) || untracked(this.loadingStory)) return;
  clearTimeout(this.timer);
  this.timer = setTimeout(() => this.autosave(fields), AUTOSAVE_MS); // 2 s after the last change
});
```

CHAPTER 7

Testing and Quality Assurance

Every part of the system has an automated test suite, and each suite must cover 100% of the code before a change is accepted. Coverage is not a goal in itself, but reaching it forced a test for every rule, error path and edge case — and doing so uncovered and fixed real defects, listed at the end of this chapter.

API

48 tests

node:test + supertest + in-memory MongoDB. 100% lines, branches and functions.

WEB APP

81 tests

Vitest + jsdom via the Angular CLI. 100% statements, branches, functions and lines.

MOBILE APP

66 tests

flutter_test unit and widget tests. 100% of lines (2,400 / 2,400).

Suite	What it checks
API	Sign-up and name rules, duplicate accounts, passwords, sessions, rate limiting, production cookie settings, CORS; publishing, drafts, scheduling, editing, deleting and ownership; sanitization; likes, views, statistics, leaderboard; migration of 2018 data; start-up and shutdown of the server; welcome e-mail.
Web app	Every page, service and component: navigation, menus, search palette shortcuts, theme, toasts and dialogs; the editor toolbar, links and focus mode; the studio's autosave, scheduling, update, unpublish, delete and leave protection; the dashboard tabs and renaming.
Mobile app	Every screen against an in-memory copy of the API: navigation, drawer and account menu, search, theme, explore, reader, leaderboard, gallery viewer, footer links, sign-in and sign-up, dashboard, and the full studio flow including date and time pickers.
Contract	Nine end-to-end checks of the mobile app's API layer against the real server: sign-up with a name, sessions that survive a restart, sanitized publishing, drafts and schedules, likes, the leaderboard and sign-out.

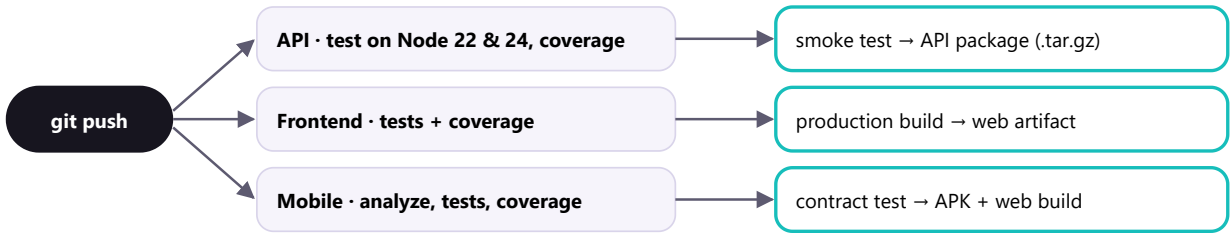


Figure 7.1 — Continuous integration on GitHub Actions

Defects found by the tests and fixed

- Opening a new story from the studio while editing a draft reused the page and overwrote the draft.

- The studio showed "Saved" while there were unsaved edits.
- The dashboard's profile panel was pushed sideways by its own background glow.
- The mobile name dialog used its text field after closing, and the leaderboard's highlighted row was painted under its panel.
- Cross-origin requests could not edit or delete stories (missing `PATCH` / `DELETE` in CORS); signing out could fail noisily when offline; shutting down the server could interrupt database operations.
- Several layouts overflowed with large text on small phones.

CHAPTER 8

Snapshots

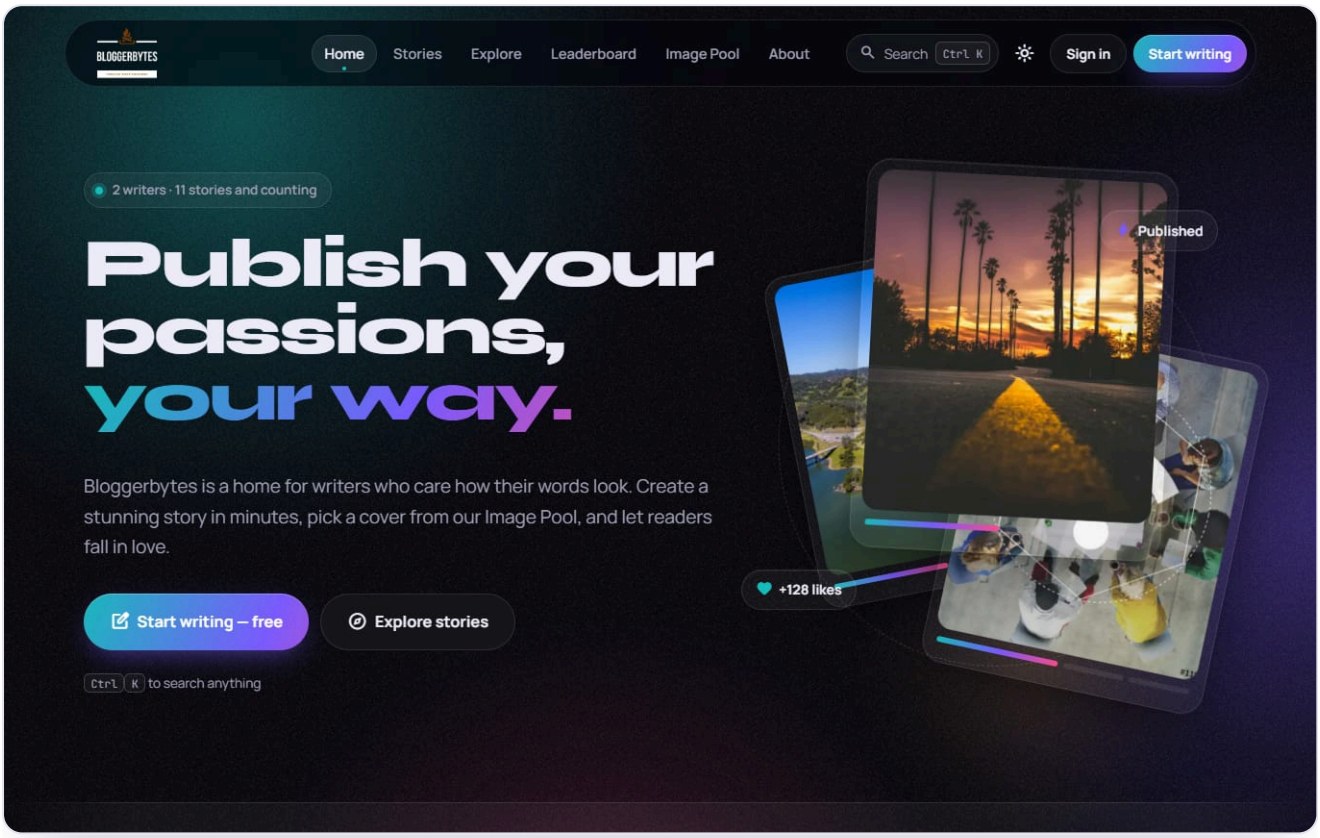


Figure 8.1 — Home page

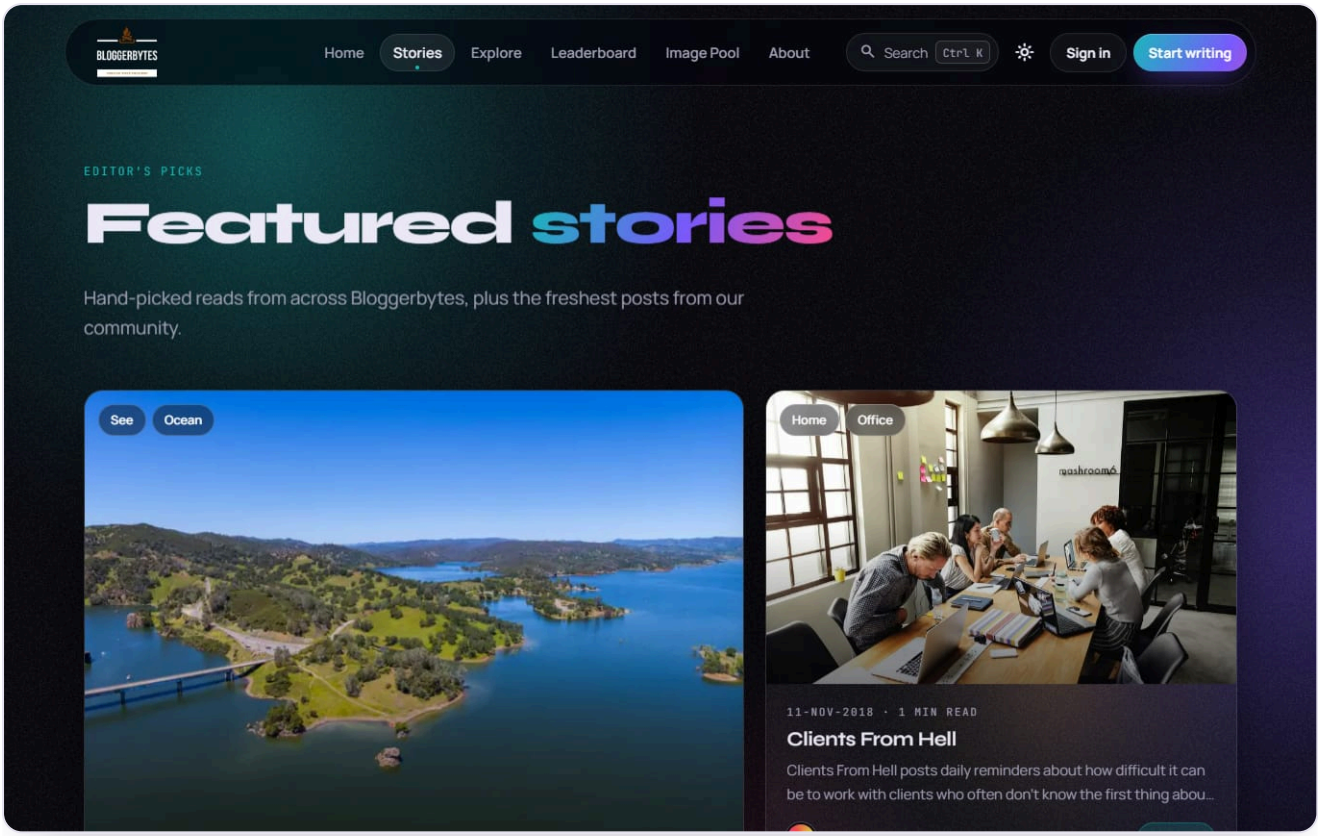


Figure 8.2 — Stories (editor's picks and latest posts)

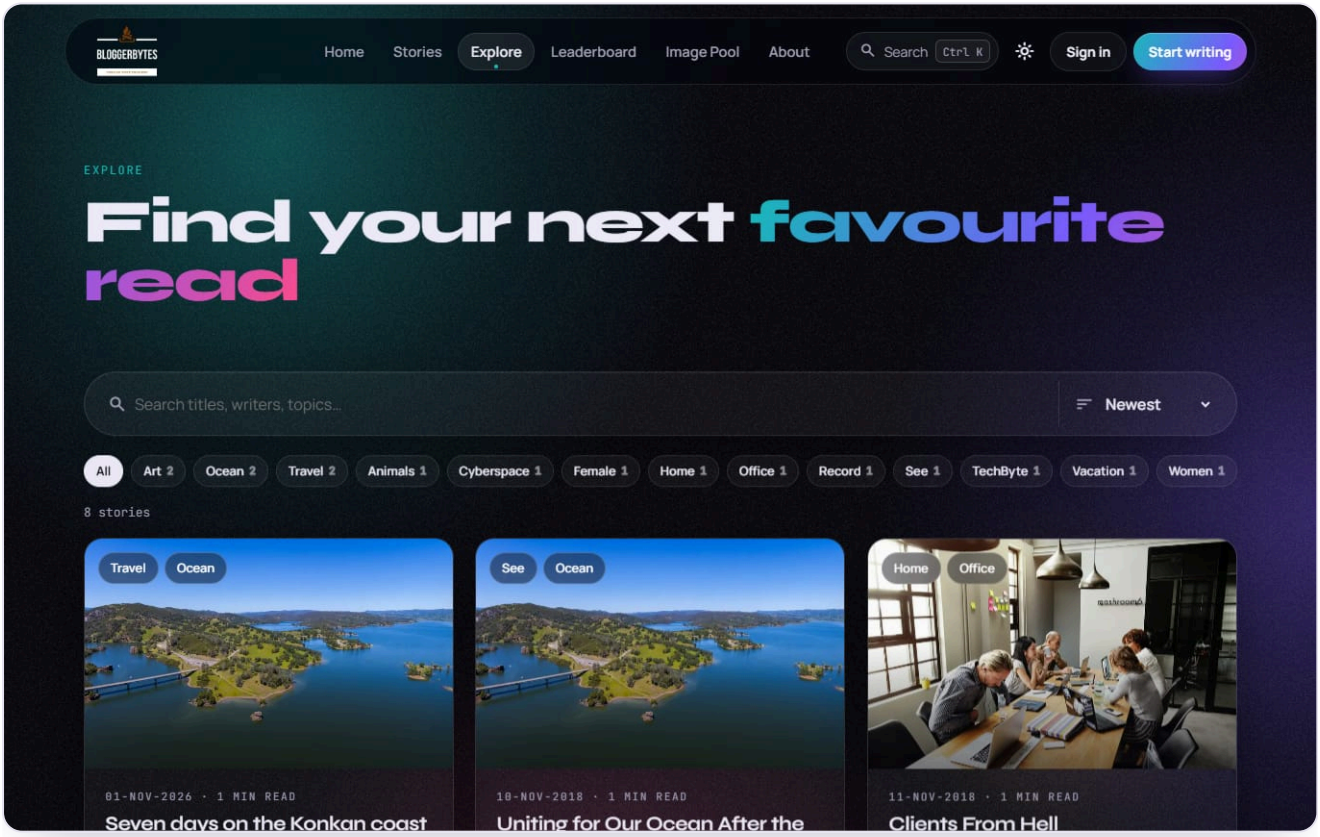


Figure 8.3 — Explore with search, topics and sorting

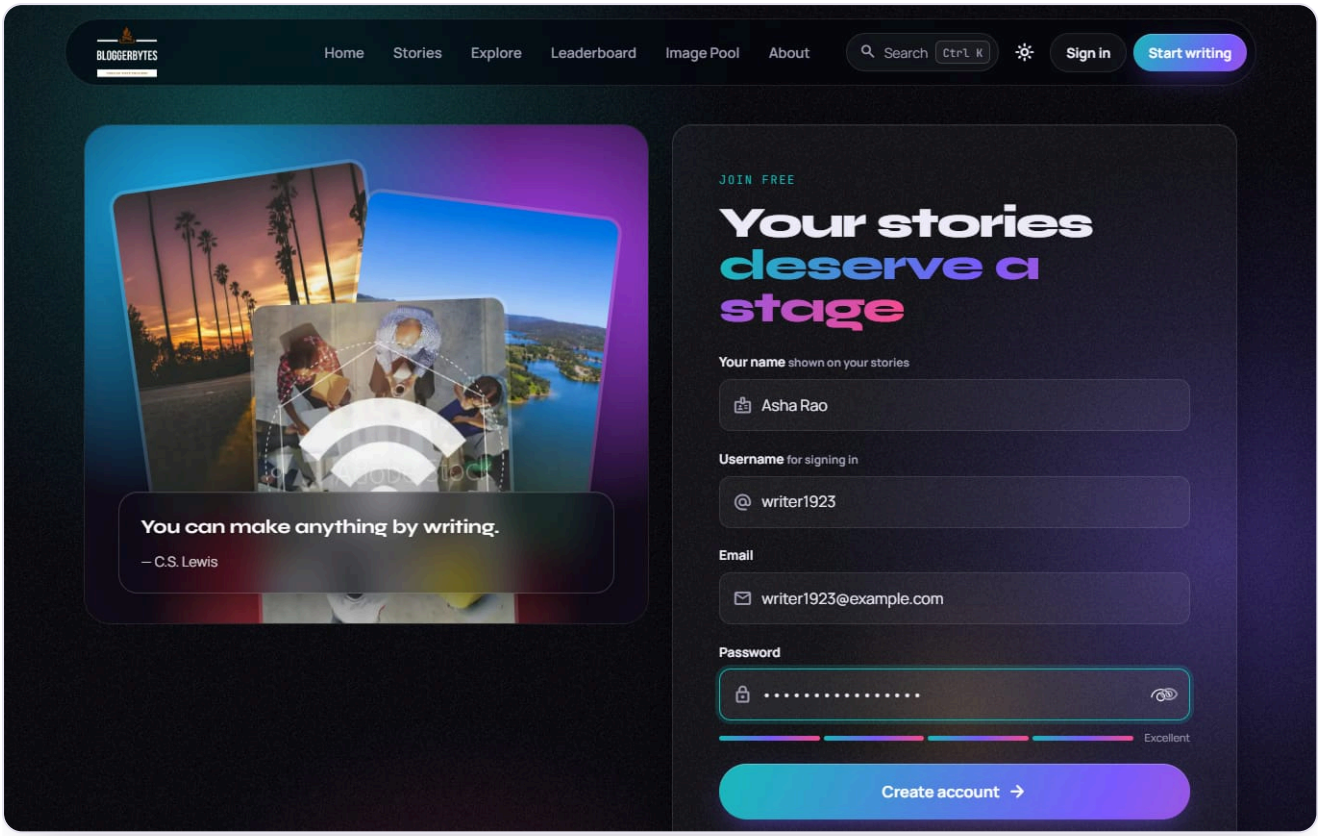


Figure 8.4 — Sign up with a display name

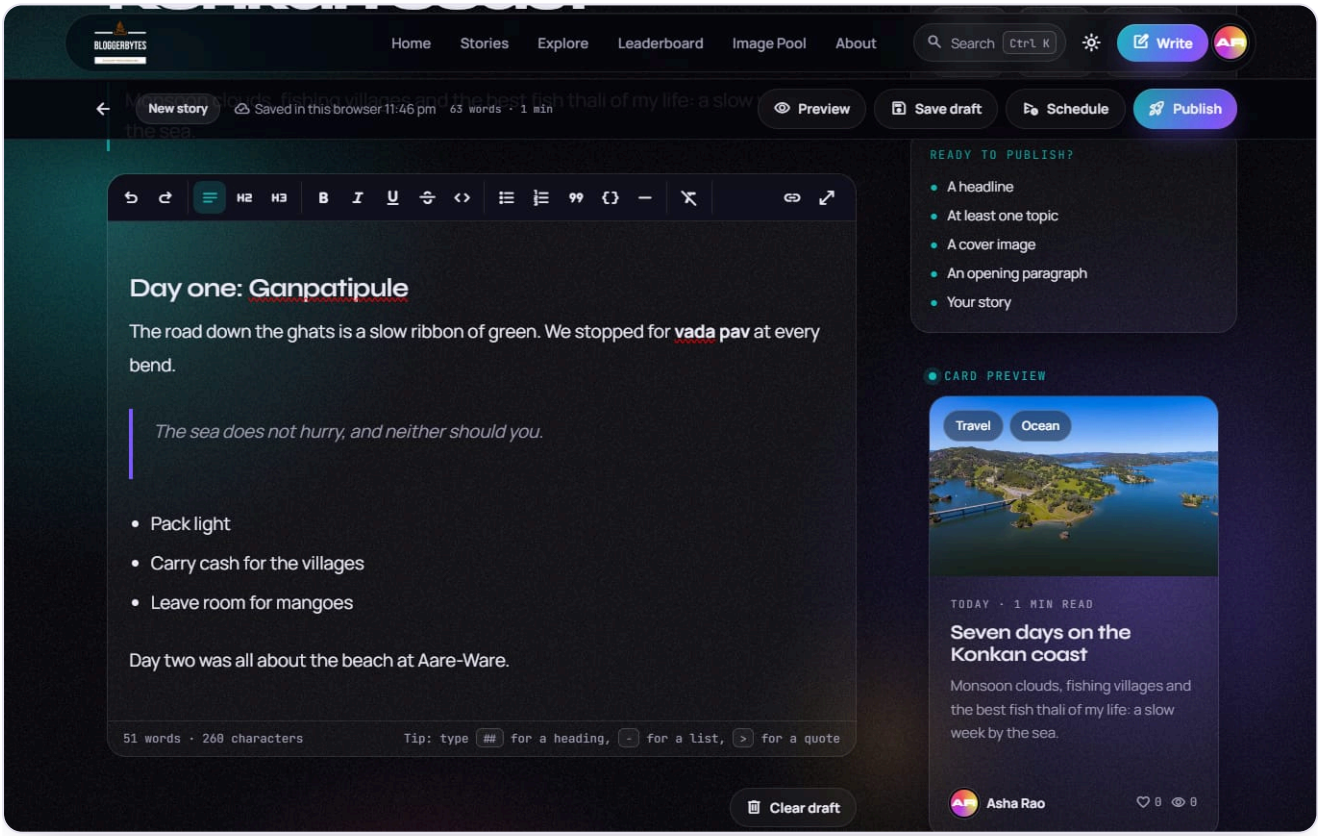


Figure 8.5 — Writing studio with the rich-text editor, checklist and card preview

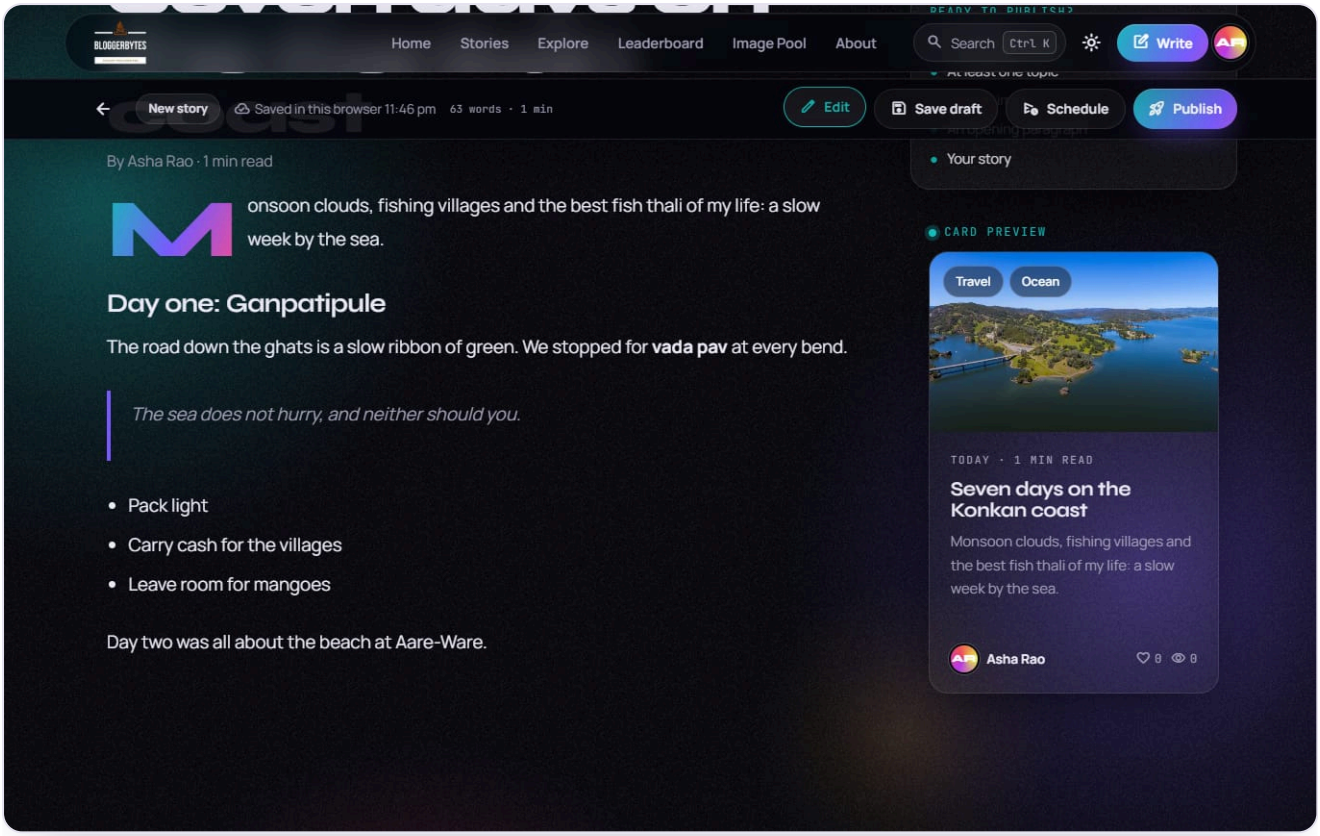


Figure 8.6 — Studio preview mode

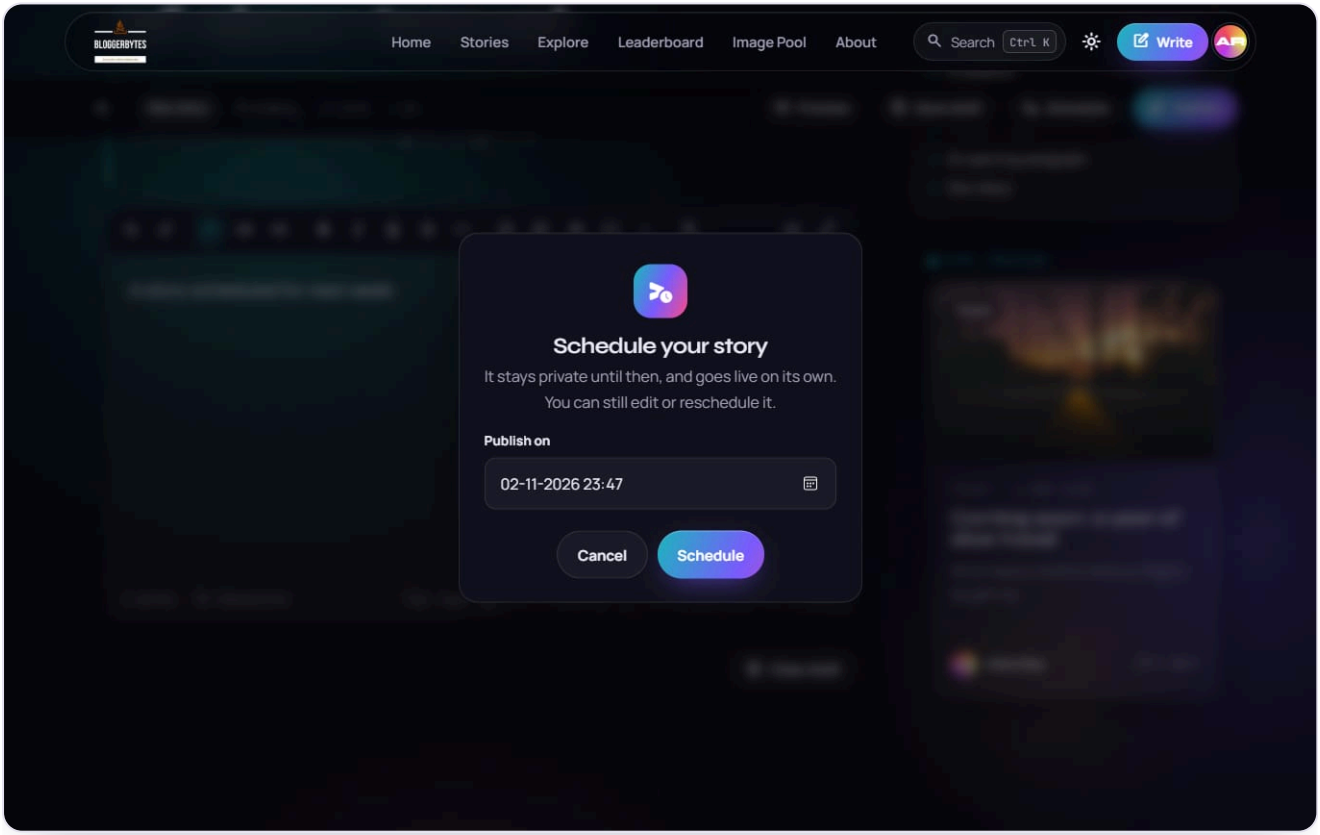


Figure 8.7 — Scheduling a story

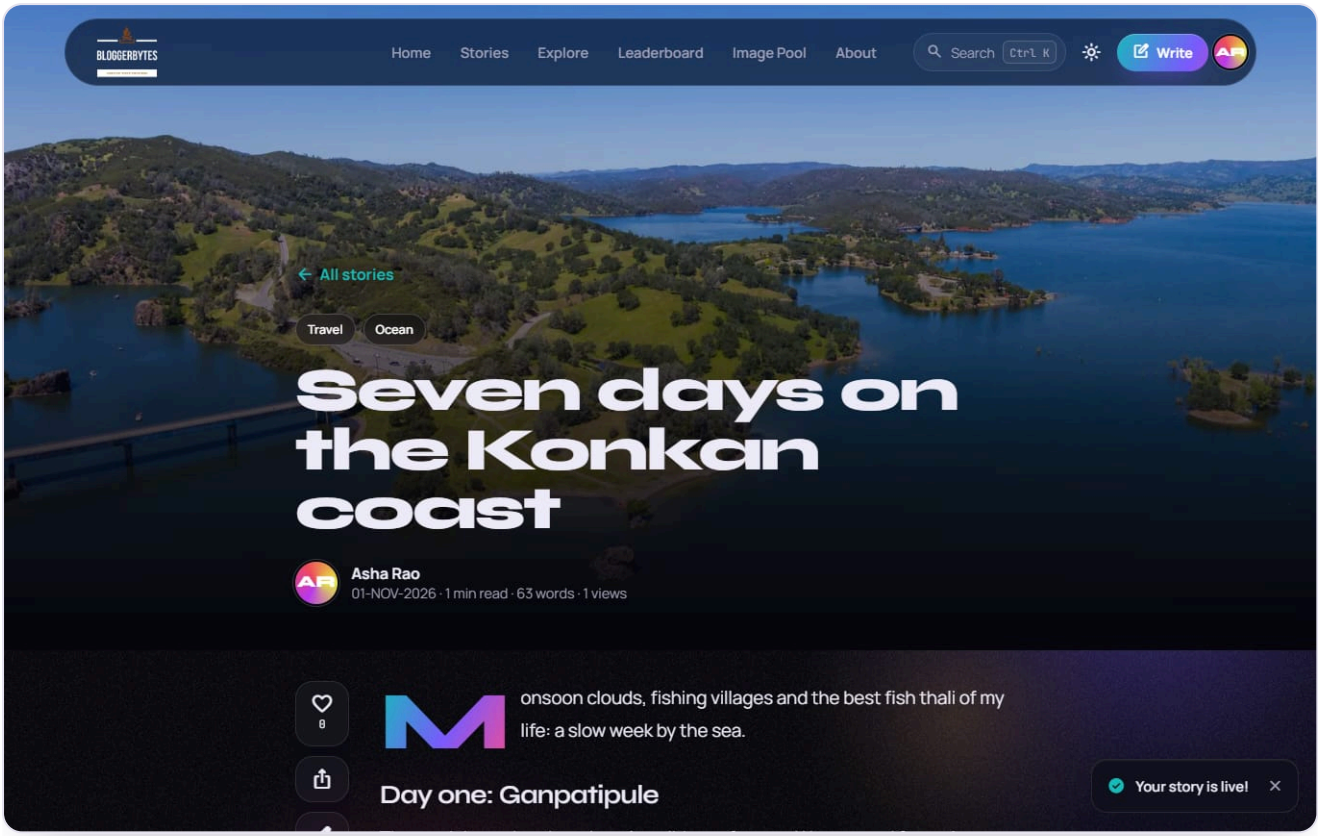


Figure 8.8 — Story reader

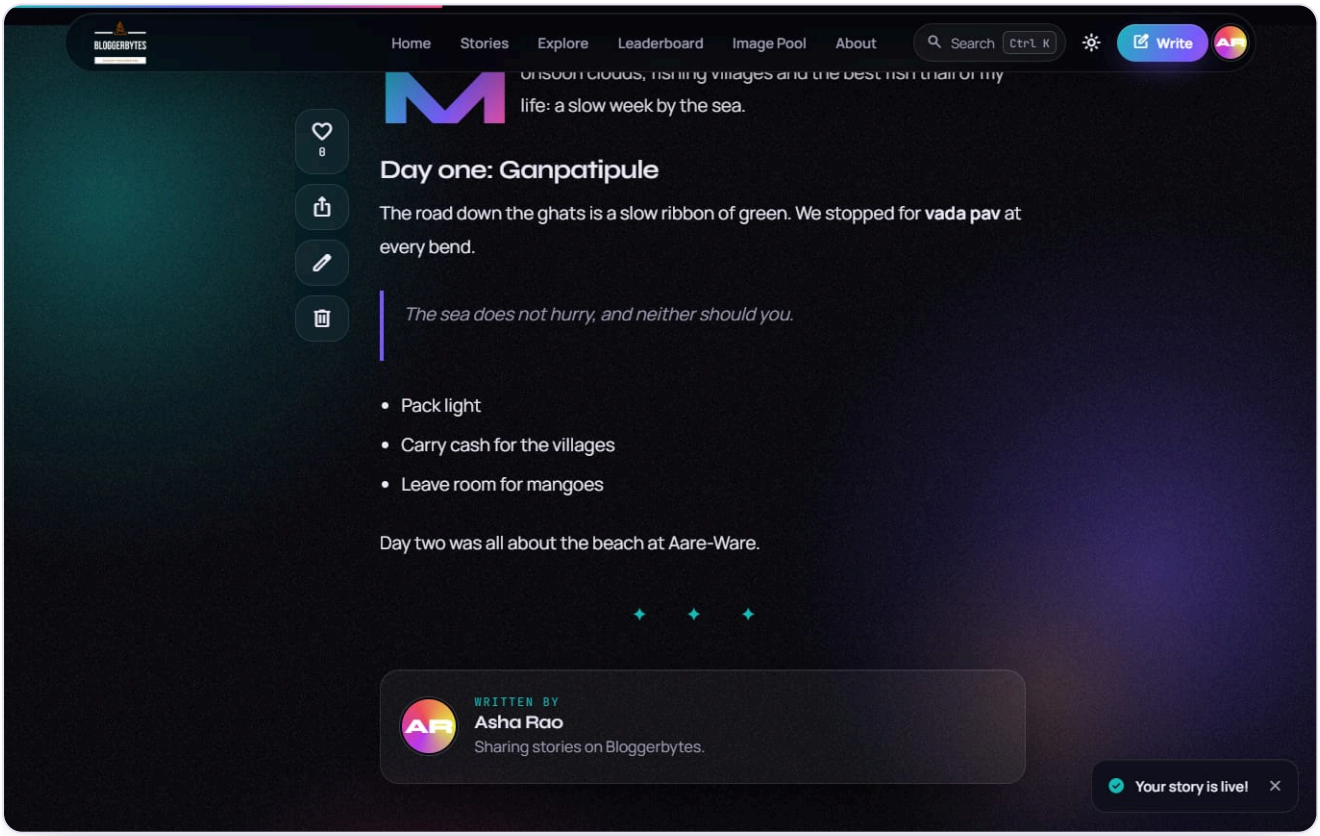


Figure 8.9 — Formatted story body with like, share, edit and delete

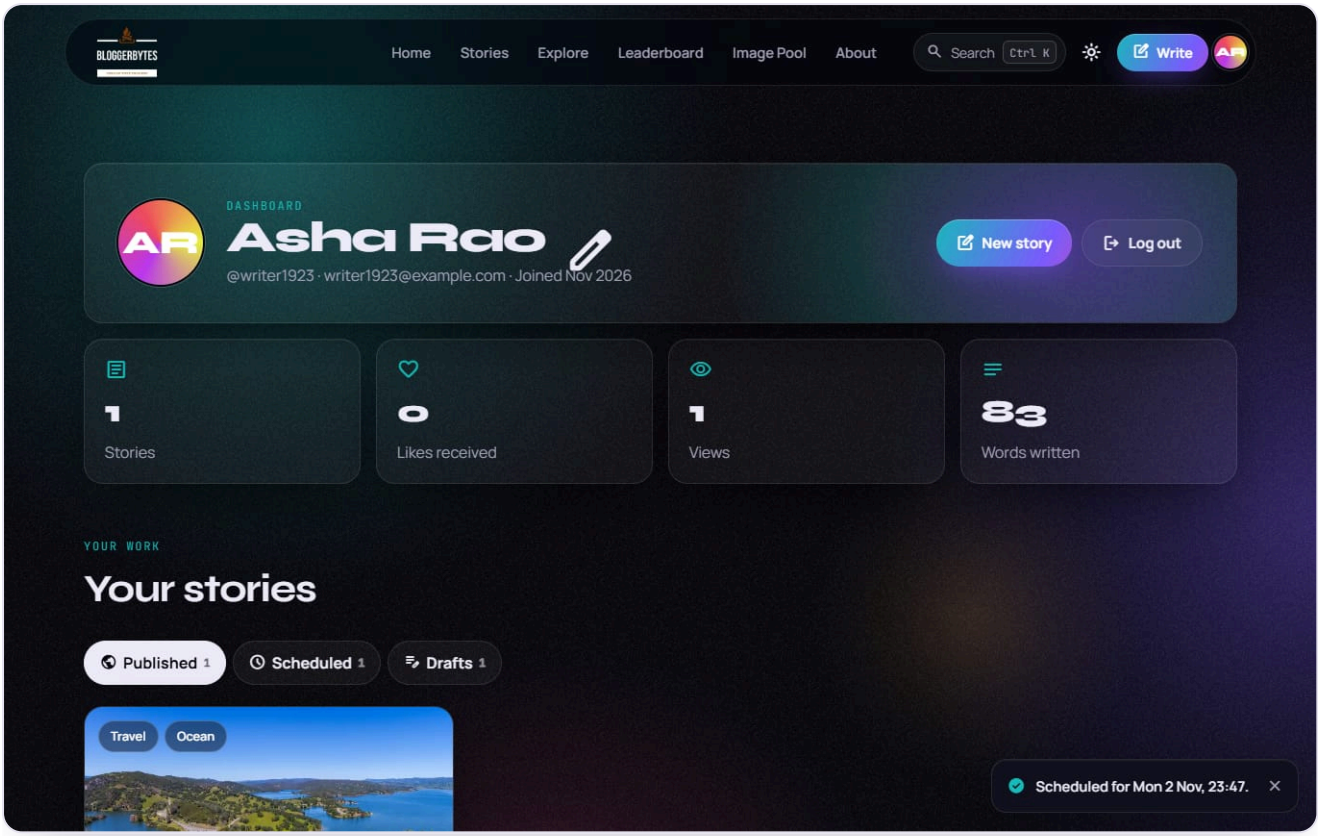


Figure 8.10 — Dashboard

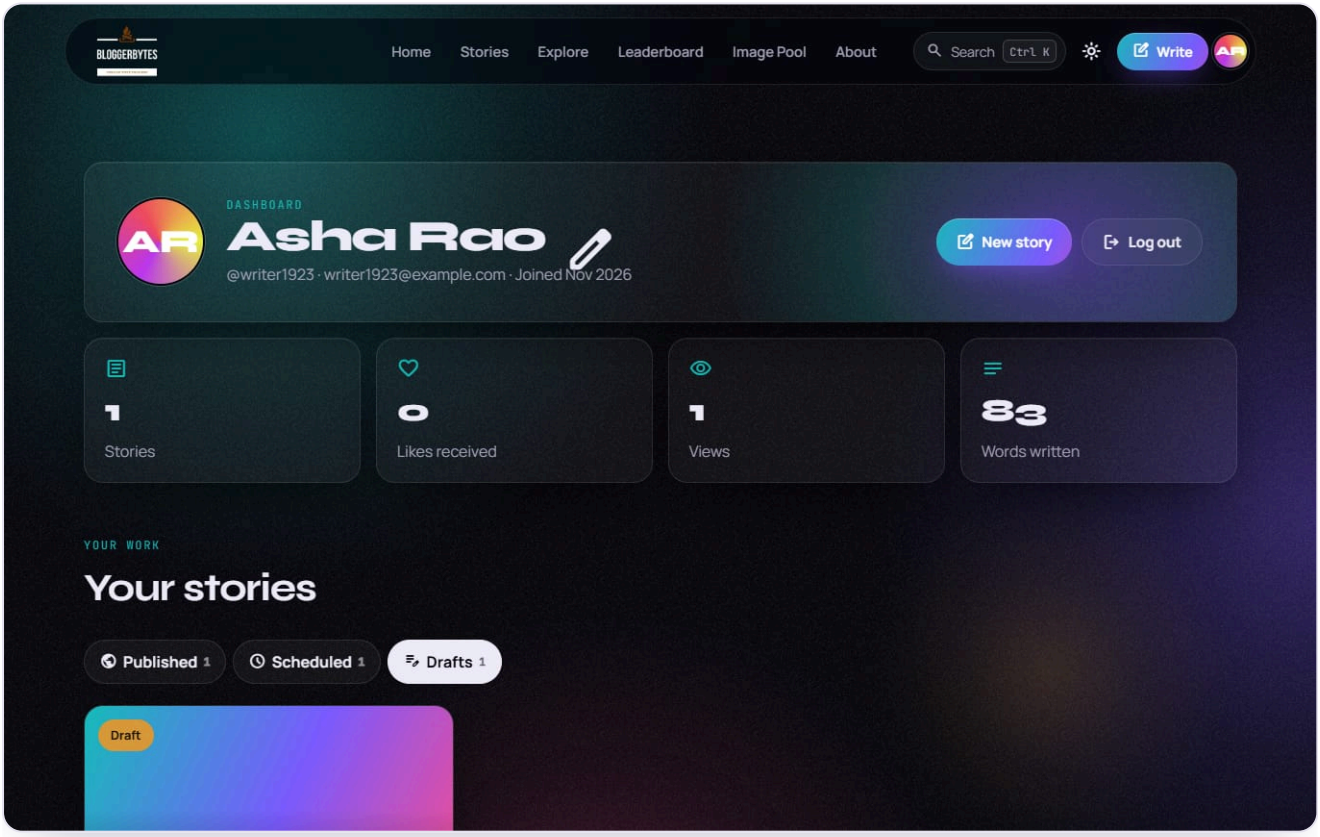


Figure 8.11 — Dashboard tabs: published, scheduled and drafts

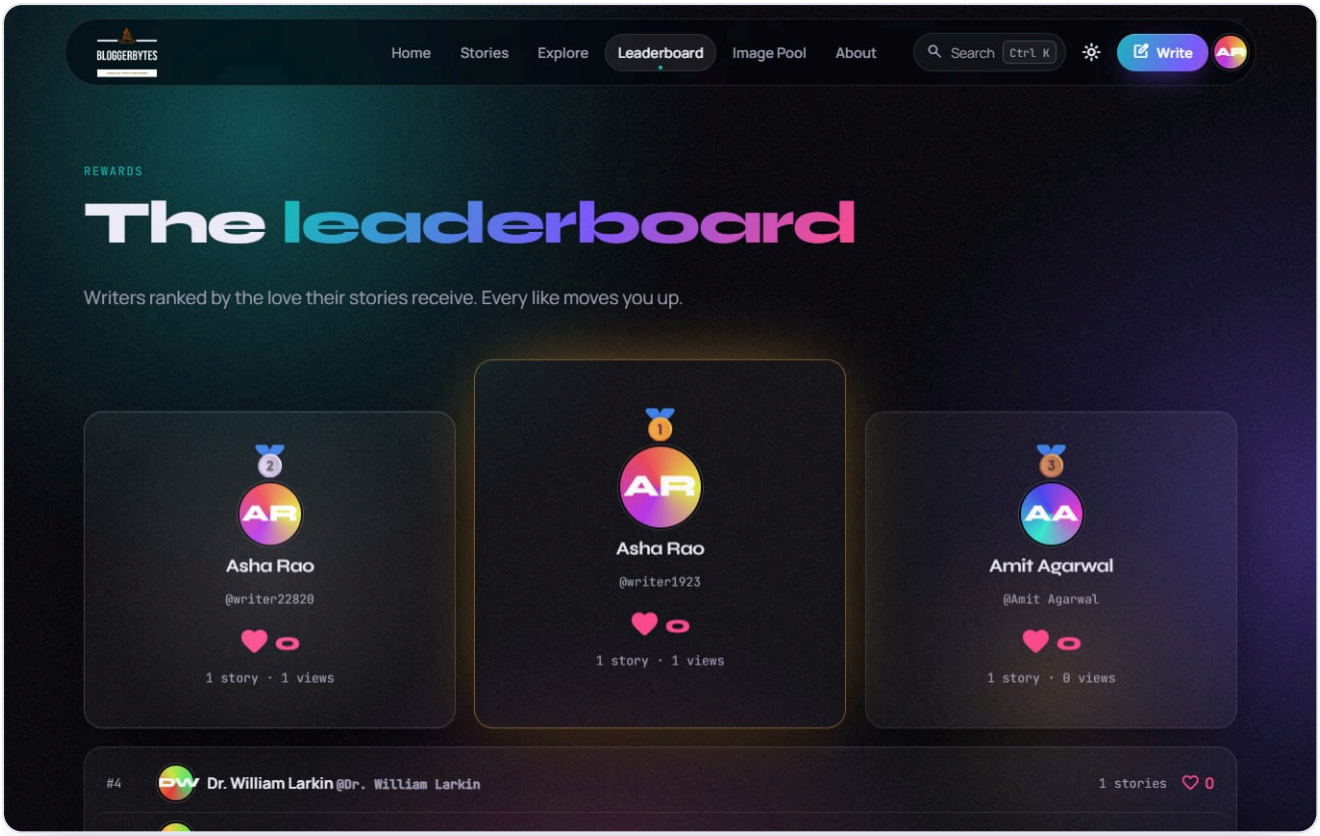


Figure 8.12 — Leaderboard

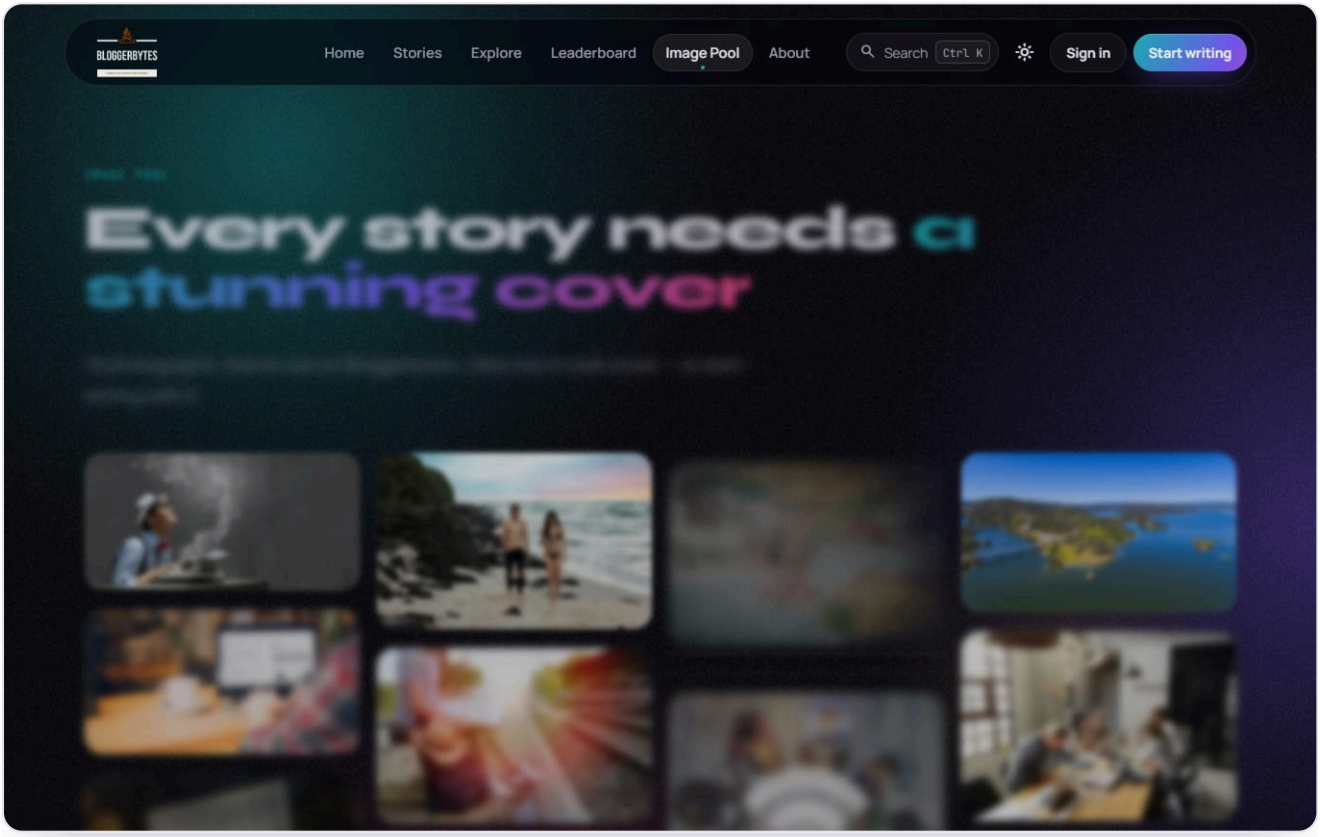


Figure 8.13 — Image Pool

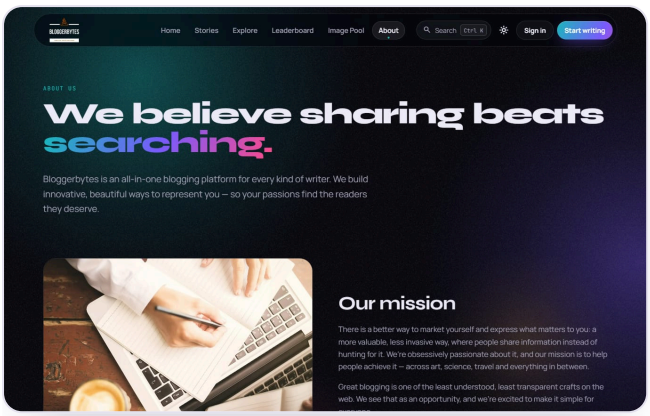


Figure 8.14a — About us

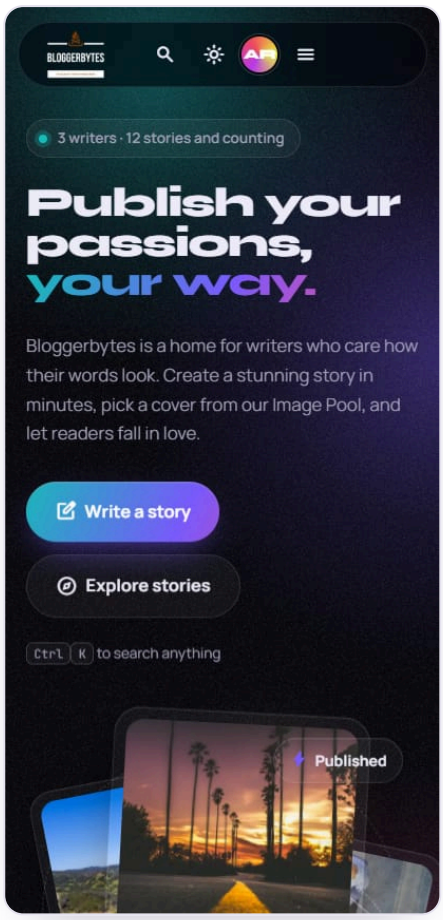


Figure 8.14b — The web app on a phone

Mobile application

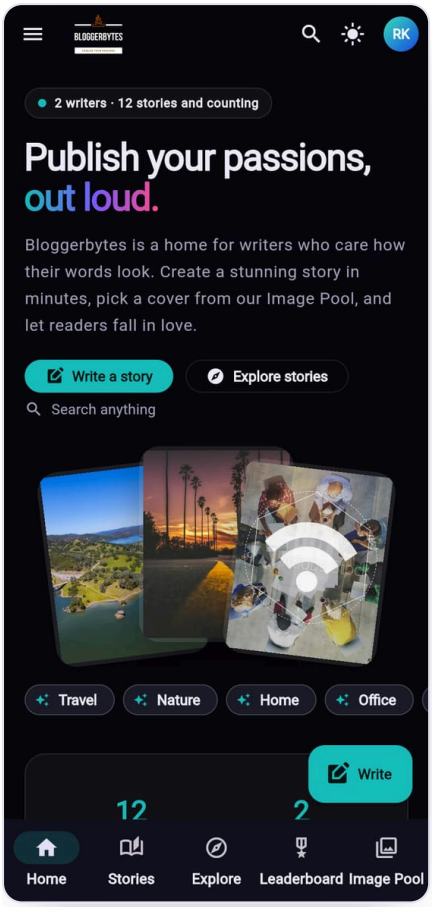


Figure 8.15 — Home

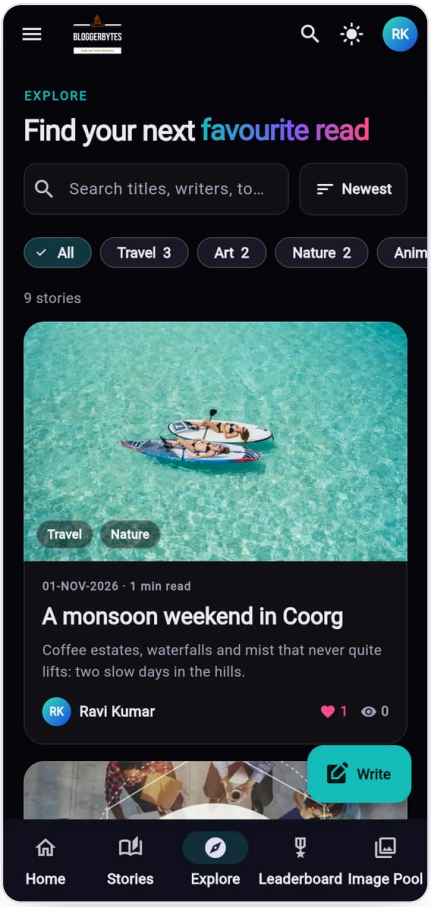


Figure 8.16 — Explore

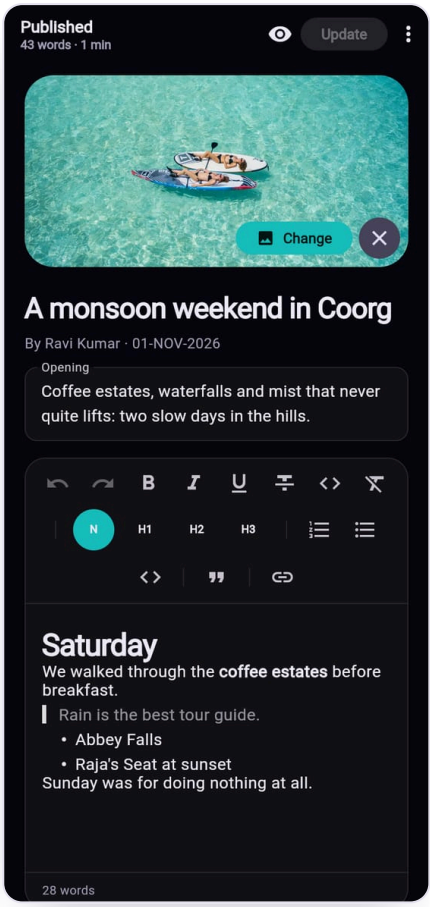


Figure 8.17 — Studio

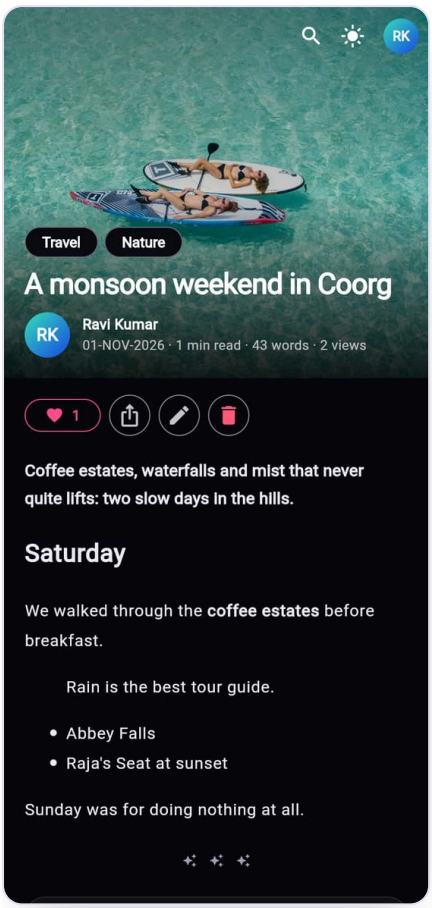


Figure 8.18 — Reader

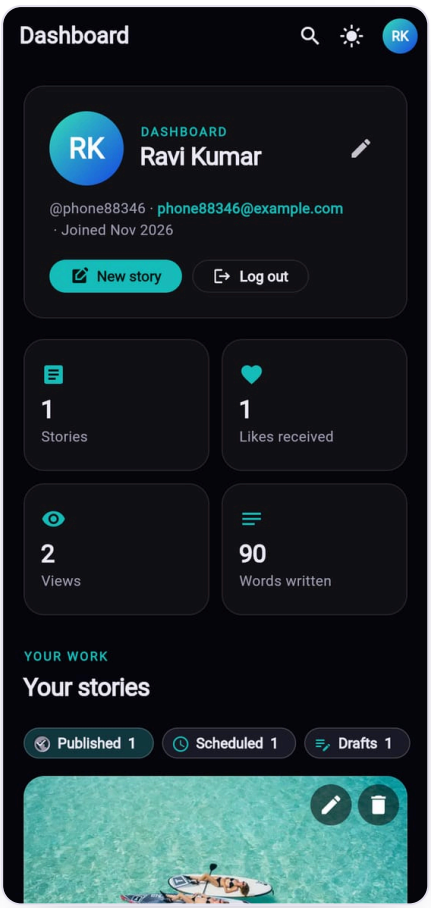


Figure 8.19 — Dashboard

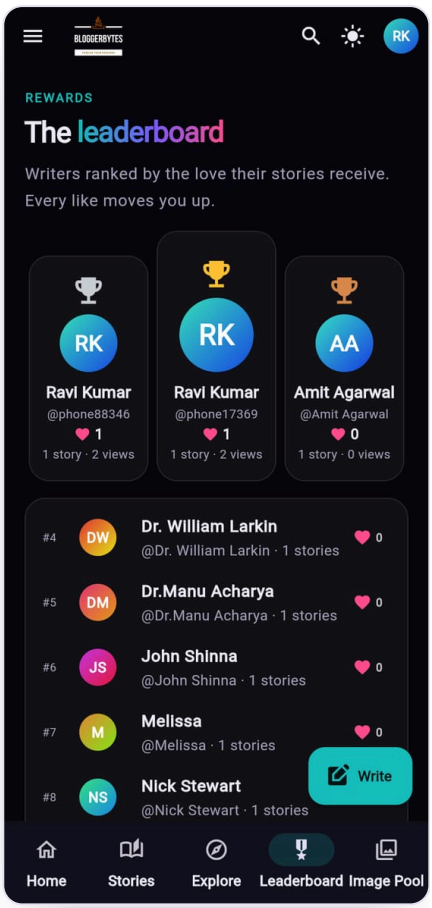


Figure 8.20 — Leaderboard

CHAPTER 9

Conclusion and Future Enhancements

Bloggerbytes 2.0 turns a student mini project into a complete product. It keeps the original idea and pages, and adds what writers and readers expect today: a real editor, drafts that never lose work, scheduled publishing, editing, names instead of logins, live rankings, instant search and a native mobile app. The original security problems are fixed by design, and three test suites at full coverage, a live contract test and continuous integration keep the system working as it grows.

The rewards concept from the original report — motivating writers by recognising the best — is now a live leaderboard driven by readers' likes.

Future enhancements

- Comments and replies on stories, with moderation.
- Follow writers and a personalised feed; e-mail or push notifications when a followed writer publishes.
- Writers' own images alongside the Image Pool (with resizing and moderation), as envisaged in the original report.
- Version history for stories and collaborative editing.
- Earned badges computed automatically from the leaderboard rules.
- Password reset by e-mail, and sign-in with Google or Apple.
- Publishing the mobile app on Google Play and the App Store.

REFERENCES

Bibliography

1. Randy Connolly, Ricardo Hoar, *Fundamentals of Web Development*, 1st Edition, Pearson Education India (ISBN 978-9332575271).
2. Robin Nixon, *Learning PHP, MySQL & JavaScript with jQuery, CSS and HTML5*, 4th Edition, O'Reilly, 2015 (ISBN 978-9352130153).
3. David Sawyer McFarland, *JavaScript & jQuery: The Missing Manual*, O'Reilly, 2014 (ISBN 978-9351108078).
4. OWASP Foundation, *OWASP Top 10* and *OWASP Cheat Sheet Series* — owasp.org.
5. Angular documentation — angular.dev.
6. Express documentation — expressjs.com; Mongoose documentation — mongoosejs.com; MongoDB manual — mongodb.com/docs.
7. TipTap documentation — tiptap.dev; ProseMirror guide — prosemirror.net.
8. Flutter documentation — docs.flutter.dev; Material Design 3 — m3.material.io.
9. MDN Web Docs — developer.mozilla.org; W3Schools — w3schools.com.
10. Original project report: *Bloggerbytes*, mini project, Department of CSE, 2018–19.